Programming with Generative AI

Viraj Kumar
Kotak-IISc AI-ML Centre

Lecture 2:

- Python REPL
- Basic operations and datatypes



A CSR Initiative by



NPTEL

Do I need to install any software?

- Recommended software:

Name	Purpose	Alternatives
Python	A useful playground (REPL), run Python code on your own device	Python shell
VSCode	A popular *Integrated Development Environment* (IDE)	OnlineGDB, PythonTutor
GitHub Copilot	A Gen AI extension for VSCode	ChatGPT, Codeium playground

- However, you can successfully complete this course using web-based alternatives

The Python REPL

- Python provides an interactive “playground” which is useful to test basic instructions:

```
>>>
```



- Let’s explore the REPL
  - Basic arithmetic

NPTEL

Basic Arithmetic Operations

Op.	Example(s)	Remarks
*	>>> 4111 * 4493 18470723	

/	>>> 3 / 2
	1.5
	>>> 4 / 2
	2.0	

//	>>> 5 // 3 1	
%	>>> 5 % 3 2	
**	>>> 5 ** 3 125	

NPTEL

Test Your Understanding

```
>>> -5 / 3
-1.6666666666666667
```

- Predict the results:

```
>>> -5 // 3
```

```
>>> -5 % 3
```

(x % y) equals
x - (x // y) * y
= -5 – (-6)

```
>>> 0 ** 0
```

```
>>> 5 % 0
ZeroDivisionError
```

NPTEL

Complex Expressions

The following table summarizes the operator precedence in Python, from highest precedence (most binding) to lowest precedence (least binding). Operators in the same box have the same precedence. Unless the syntax is explicitly given, operators are binary. Operators in the same box group left to right (except for exponentiation and conditional expressions, which group from right to left).

- What about: `>>> 2 ** 3 ** 4` $(2^3)^4 = 2^{3\times4} = 2^{12}$ or $2^{(3^4)} = 2^{81}$

Operator	Description
(. . .)	Parentheses
**	Exponentiation
+x, -x	Unary operators
*, /, //, %	Binary operators
+, -	Binary operators

NPTEL

Are there any limits?

Numeric Types — `int`, `float`, `complex`

There are three distinct numeric types: *integers*, *floating point numbers*, and *complex numbers*. In addition, Booleans are a subtype of integers. Integers have unlimited precision. Floating point numbers are usually implemented using `double` in C; information about the precision and internal representation of floating point numbers for the machine on which your program is running is available in `sys.float_info`. Complex numbers have a real

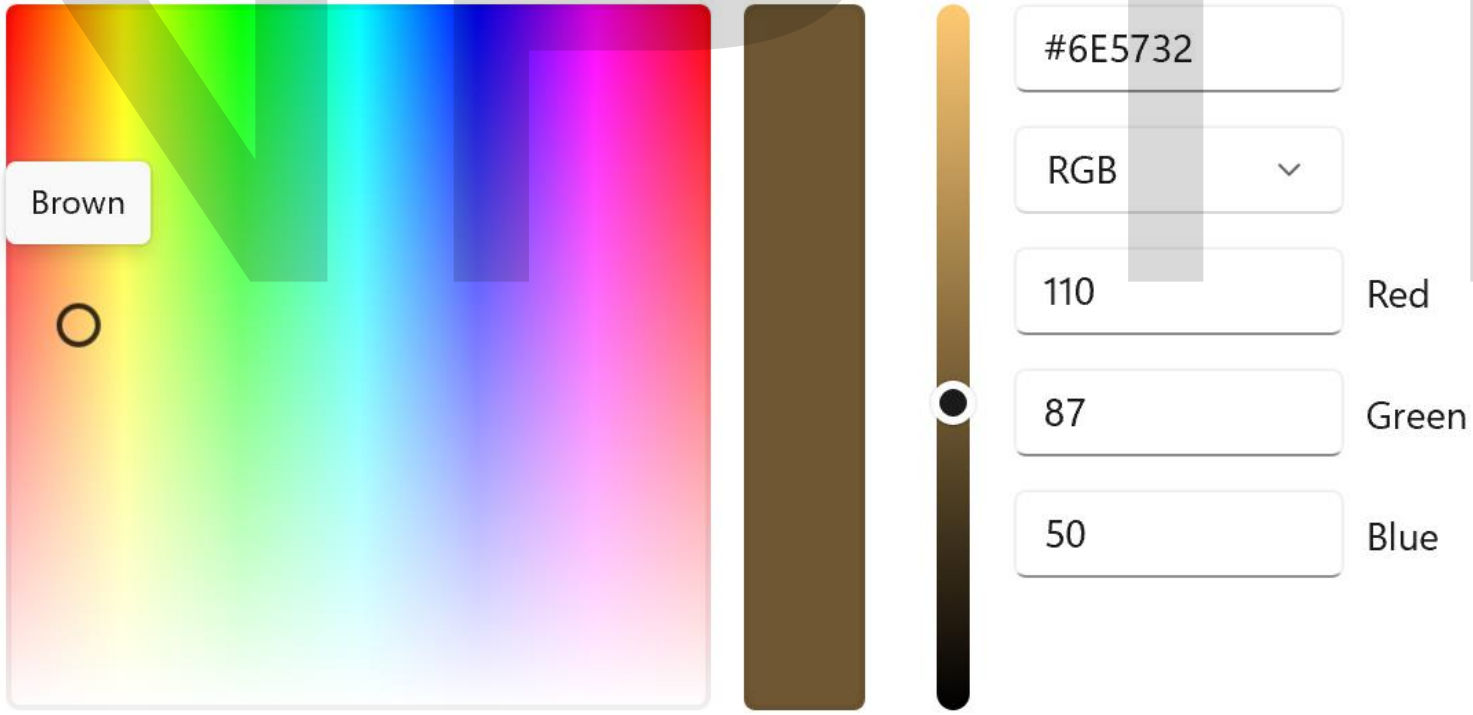
- This is **impossible**! (Limited memory)
  - More accurately: **Arbitrary precision** for **`int`**
  - What about **`float`**?
- Let's find out!
  - Warning: REPL error messages are written by experts for experts

NPTEL

Everything is **bits** (**bi**nary digi**ts**)

```
>>> int('101', 2)
5
>>> int('6E', 16)
110
```

- We count in base 10 (decimal digits):
  - 0, 1, …, 9, 10, … 99, 100, 101, …
- Computers count in base 2 (binary digits):
  - 0, 1, 10, 11, 100, 101, …



NPTEL

Test ~~Your~~ Our Friend's Understanding

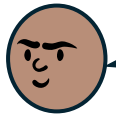
- My computer represents each colour using 24 bits.

$$2^{24} = 2^{4+20} = (2^4) \times 1\text{M}$$



You can have about 16 million colours!

$$2^{(3^4)} = 2^{81}$$

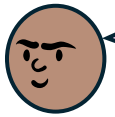


It takes 81 bits to represent this number!

```
>>> 2 ** 3 ** 4
2417851639229258349412352
>>> (2 ** 3 ** 4).bit_length()
82
```

method

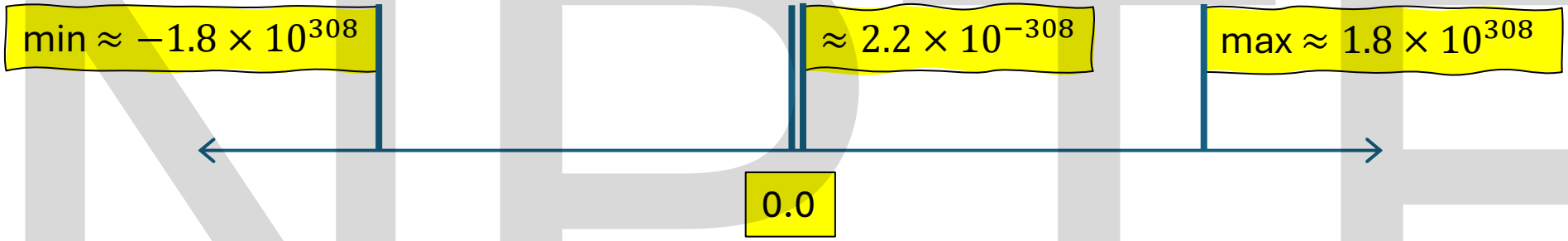
- The programming language Java represents `int` using 32 bits.



This Python `int` cannot be represented as a Java `int`

`float` limits

- Real number line (infinite)



- **Key point**: Many real numbers **cannot** be represented as `float` in Python
  - E.g., 0.1 is represented as $\frac{3602879701896397}{36028797018963968}$

```
>>> 0.1 * 3 == 0.3
False
```

Python **objects**

- A Python programmer must represent real-world entities as Python **objects**
  - This is an **abstract** representation
  - It hides/approximates certain aspects of real-world entities (e.g., `0.1` $\approx \frac{1}{10}$)
- So far, we have seen objects of two **types**:
  - `int` (e.g., `312`)
  - the expression `3 * 1 // 2` evaluates to an `int`)
  - `float` (e.g., `1.5`)
  - the expression `3 * 1 / 2` evaluates to a `float`)

NPTEL

Another useful type: `str`

```
>>>ord('A')
65
>>>ord('$')
36
>>>ord('₹')
```

- A `str` (string) is an abstract representation of textual data

```
>>>"""August 15
...1947"""
'August 15\n1947'
```

- Internally, every letter of a `str` is represented as an integer
  - The letter `\` is special (“escape” symbol)

```
>>>ord('\n')
10
```

NPTEL

Summary

- The **REPL** is a useful “playground”
  - But we will soon outgrow it
- Everything is represented as **bits** (0’s and 1’s)
  - Integers in base 2 (binary)
  - Colours, letters, … as integers
- Not everything can be represented
  - E.g., most real numbers
- Can lead to surprises and disasters:
  - Patriot missile failure (28 deaths)
  - Ariane 5 explosion (USD 7 billion loss)

NPTEL